30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 11

Network Layer — Part 1: HTTP, Retrofit, OkHttp, Ktor & Resilient Calls



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 11

Why the network layer matters in 2026

Almost every screen waits on a server — and mobile networks are slow, flaky and sometimes lie ("connected", but no internet). Interviewers use this topic to see whether you design for **failure**, not just for the happy path:

- **Retrofit + OkHttp** is still the Android default; the **Ktor client** is the KMP choice
- **kotlinx.serialization** replaced Gson in new Kotlin code
- `suspend` API calls: no call adapters, main-safe, cancellable
- Resilience is a design task: timeouts, retries with backoff, typed errors
- HTTP caching and connectivity are about **correctness**, not only speed

Level	Questions
Junior	HTTP basics · Retrofit + OkHttp · Serialization
Mid	Interceptors · Timeouts & retries · Error modeling · HTTP caching
Senior	Ktor & KMP · Connectivity · Cancellation & concurrency
Architect	Network module design · Resilience at scale · Evolving APIs



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: the path of one request

```
Repository      suspend fun user(id): User
    │
    ▼
Retrofit/Ktor   Kotlin call → HTTP request · JSON → DTO
    │
    ▼
OkHttp          interceptors · cache · redirects · retries
    │
    ▼
Connections     pool · DNS · TLS · HTTP/2
    │
    ▼
Server  ──►  status + headers + body  ──►  back up
```

- **Retrofit / Ktor** *describe* the API; **OkHttp** (or another engine) *executes* it
- **Your code** owns the policy: which errors to retry, what to cache, what the user sees
- DTOs stop at the data layer — the UI never sees a network model

THE QUESTION BEHIND EVERY NETWORK QUESTION

"What happens when this call is slow, fails halfway, or runs twice?" Answer that and you've answered the interviewer.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 11



Junior

HTTP basics · Retrofit + OkHttp · Serialization



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. HTTP methods: safe vs idempotent

Method	Safe	Idempotent	Typical use
`GET` / `HEAD`	Yes	Yes	Read a resource
`POST`	No	No	Create, or run a command
`PUT`	No	**Yes**	Replace a resource completely
`PATCH`	No	Not guaranteed	Partial update
`DELETE`	No	**Yes**	Remove a resource

- **Safe:** the client doesn't ask for any state change on the server
- **Idempotent:** sending the same request N times has the same *effect* as sending it once — the *response* may differ (a second `DELETE` can return `404`)

INTERVIEW TRAP

"POST and DELETE are not idempotent." — Half wrong, and last year's version of this challenge said it. By the HTTP spec, `DELETE` **is** idempotent; `POST` is not. And idempotent ≠ "doesn't change state": `PUT` changes state, but repeating it changes nothing more.



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Status codes you must react to

Code	Meaning	Client reaction
`200` / `201` / `204`	OK / Created / No Content	Use the body — `204` has **none**
`400` / `422`	Invalid request	Show the validation error; don't retry
`401`	Not authenticated	Refresh the token, then retry once (Day 12)
`403`	Authenticated, not allowed	Don't refresh; show "no access"
`408` / `429`	Timeout / too many requests	Retry later; honor `Retry-After`
`500` / `502` / `503` / `504`	Server or gateway failure	Retry with backoff — idempotent calls only

INTERVIEW TRAP

"401 and 403 mean the same thing." — `401`: we don't know who you are (missing or expired credentials). `403`: we know, and the answer is no. Refreshing a token on `403` never helps.

FOLLOW-UP

"Why does HTTP/2 matter on mobile?" — Many requests share **one** connection (multiplexing), so fewer TLS handshakes. OkHttp negotiates it automatically; HTTP/3 (QUIC) needs Cronet.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Retrofit + OkHttp: who does what

```
interface UserApi {
    @GET("users/{id}")
    suspend fun user(@Path("id") id: String): UserDto

    @POST("users")
    suspend fun create(@Body body: NewUser): Response<UserDto>
}

val contentType = "application/json".toMediaType()
val retrofit = Retrofit.Builder()
    .baseUrl("https://api.example.com/v1/")
    .client(okHttpClient) // one shared instance
    .addConverterFactory(json.asConverterFactory(contentType))
    .build()

val api: UserApi = retrofit.create()
```

- **Retrofit** turns the interface into HTTP calls and converts bodies
- **OkHttp** does the real work: connection pool, HTTP/2, TLS, transparent gzip, cache, redirects, interceptors



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. The details interviewers probe

- `suspend fun user(): UserDto` — throws `HttpException` on non-2xx, `IOException` on network failure
- `suspend fun create(): Response<UserDto>` — no exception for HTTP errors: check `isSuccessful`, `code()`, `errorBody()`. Network failures still throw.
- One `OkHttpClient` for the whole app — each client has its own connection pool and thread pool

INTERVIEW TRAP

"You need `CoroutineCallAdapterFactory` for suspend functions." — Outdated (last year's deck said it too). Retrofit supports `suspend` natively since **2.6**. That adapter was for returning `Deferred`.

INTERVIEW TRAP

`baseUrl` must end with `/`, and `@GET("/users")` with a leading slash is resolved from the **host root** — it silently drops `/v1/`. Use relative paths.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Serialization: why kotlinx.serialization

	kotlinx.serialization	Moshi	Gson
How	Compiler plugin, no reflection	KSP codegen or reflection	Reflection
Kotlin-aware	Defaults, nullability	Yes	**No**
KMP	Yes	No	No
Status	Default for new code	Maintained	Maintenance mode

```kotlin
@Serializable
data class UserDto(
    val id: String,
    @SerialName("display_name") val name: String,
    val avatarUrl: String? = null,
)
```

INTERVIEW TRAP

"Gson works fine with Kotlin." — Gson creates objects **without calling the constructor**: default values are ignored and a non-null `String` can end up `null`, crashing far from the parsing code.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Configure the parser for the real world

```
val json = Json {
    ignoreUnknownKeys = true    // server adds a field → no crash
    coerceInputValues = true    // null or unknown enum → default
    explicitNulls = false       // missing nullable field → null
}
```

- Default `Json` **throws** on unknown keys — one new backend field can break every installed app
- Model what the server *may* omit as nullable or with a default
- Map DTOs to domain models at the data-layer boundary (Day 3)

PARSE AT THE EDGE, VALIDATE ONCE

A DTO describes the wire format; a domain model describes your rules. Keep them separate so a backend rename touches one mapper, not twenty screens.

FOLLOW-UP

"Where does parsing run?" — Retrofit's converters run on OkHttp's background threads, so a big response doesn't block the main thread.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 11

Mid-Level

Interceptors · Timeouts & retries · Error modeling · HTTP caching



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. The interceptor chain

```
call ─▶ application interceptors   once per call
        RetryAndFollowUp            redirects · retries
        Bridge                      gzip · cookies · headers
        Cache                       may answer from disk
        Connect                     pool · TLS
     ─▶ network interceptors        once per network hop
        CallServer                  bytes on the wire
```

	`addInterceptor`	`addNetworkInterceptor`
Called	Once per call, even for cache hits	Per network request: each redirect and retry; not for cache hits
Sees	The request your code built	The final request (`Host`, `Accept-Encoding: gzip`) and the connection
`proceed()`	Zero times (short-circuit) or several	Exactly once
Use for	Auth header, logging, app-level policy	Network metrics, wire-level debugging



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. An auth-header interceptor

```
class AuthHeaderInterceptor(
    private val tokens: TokenStore,
) : Interceptor {
    override fun intercept(chain: Interceptor.Chain): Response {
        val token = tokens.accessToken()
            ?: return chain.proceed(chain.request())
        val request = chain.request().newBuilder()
            .header("Authorization", "Bearer $token")
            .build()
        return chain.proceed(request)
    }
}
```

- Interceptors run on OkHttp's threads and are **blocking** — keep them fast
- `header()` replaces a header; `addHeader()` appends a duplicate

INTERVIEW TRAP

`HttpLoggingInterceptor` at `BODY` level in release writes tokens and personal data to Logcat. Use `NONE` in release and `redactHeader("Authorization")` in debug.

INTERVIEW TRAP

Last year's deck added `CertificatePinner` as a network interceptor and called `Authenticator` an interceptor. Neither is: they are set with `certificatePinner()` and `authenticator()` (Day 12).



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Timeouts: what each one really covers

OkHttp timeout	Default	Covers
`connectTimeout`	10 s	Opening the TCP connection to the host
`readTimeout`	10 s	Max **gap** between bytes while reading — not the total
`writeTimeout`	10 s	Max gap between writes of the request
`callTimeout`	none	The **whole** call: DNS, connect, send, server time, body, redirects, retries

- A response trickling one byte every 9 s never trips `readTimeout` — only `callTimeout` bounds what the user waits
- In coroutines, `withTimeout { api.user(id) }` works too: cancellation cancels the OkHttp call

INTERVIEW TRAP

"Timeouts stop the UI thread from freezing." — Last year's claim. Network calls never run on the main thread (Android throws `NetworkOnMainThreadException`). Timeouts bound how long a **background** call — and the user — waits.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Retries with exponential backoff and jitter

```
suspend fun <T> withRetry(
    maxAttempts: Int = 3,
    baseDelayMs: Long = 500,
    maxDelayMs: Long = 8_000,
    isTransient: (Throwable) -> Boolean,
    block: suspend () -> T,
): T {
    var cap = baseDelayMs
    repeat(maxAttempts - 1) {
        try {
            return block()
        } catch (e: CancellationException) {
            throw e
        } catch (e: Exception) {
            if (!isTransient(e)) throw e
        }
        delay(Random.nextLong(cap + 1)) // full jitter
        cap = (cap * 2).coerceAtMost(maxDelayMs)
    }
    return block()
}
```

- Retry only **transient** failures: `IOException`, `408`, `429`, `5xx` — and only **idempotent** calls
- Jitter spreads clients out, so thousands of phones don't retry in the same second



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. What OkHttp retries by itself

INTERVIEW TRAP

"OkHttp automatically retries transient failures like DNS problems." — Not in the sense last year's deck meant. `retryOnConnectionFailure` (on by default) silently recovers from a few **connection** problems — a stale pooled connection, an unreachable route when other IPs exist. It never retries a `503`, and it has no backoff.

INTERVIEW TRAP

Retrying a `POST /payments` after a timeout. The server may have charged the card and only the response was lost. Retry non-idempotent calls only with an **idempotency key** (Q12).

FOLLOW-UP

"The server sends `Retry-After`. Now what?" — It wins over your backoff formula. It can be seconds or an HTTP date; honor both.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Model errors once, at the boundary

```
suspend fun <T> apiCall(block: suspend () -> T): ApiResult<T> =
    try {
        ApiResult.Ok(block())
    } catch (e: CancellationException) {
        throw e
    } catch (e: SocketTimeoutException) {
        ApiResult.Err(ApiError.Timeout)
    } catch (e: IOException) {
        ApiResult.Err(ApiError.Network)
    } catch (e: HttpException) {
        val body = e.response()?.errorBody()?.string()
        ApiResult.Err(ApiError.Http(e.code(), body))
    } catch (e: SerializationException) {
        ApiResult.Err(ApiError.Parsing(e))
    }
```

- Order matters: `SocketTimeoutException` **is** an `IOException` — catch it first
- `ApiError` is a sealed interface; the ViewModel maps it to a message and never sees `HttpException`

INTERVIEW TRAP

Classifying errors by `e.message` ("timeout", "No address associated..."), as last year's deck suggested. Messages change between versions and locales — use exception **types** and status codes.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. From error to user experience

Error	User sees	Retry?
Network (`UnknownHostException`, `ConnectException`)	"You're offline"	Yes — when back online
Timeout	"Taking longer than usual"	Yes, if idempotent
`401`	Nothing — silent refresh (Day 12)	Once, after refresh
`400` / `404` / `422`	Specific message from the error body	No
`429` / `503`	"Busy, trying again…"	Yes, after `Retry-After`
Other `5xx`	"Something went wrong on our side"	Yes, with backoff
Parsing	Generic message; log a non-fatal	No — it's a bug

INTERVIEW TRAP

"CancellationException is just another error — log it." — Last year's advice. It is **not** a failure: rethrow it, or structured cancellation breaks. The same goes for `runCatching` around suspend calls.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. HTTP caching: the headers that decide

Header	Meaning
`Cache-Control: max-age=60`	Fresh for 60 s — served from disk, no network
`Cache-Control: no-cache`	May be stored, but must be **revalidated** before every use
`Cache-Control: no-store`	Never write it to disk
`ETag` → `If-None-Match`	Revalidate by version; `304` means "your copy is fine"
`Last-Modified` → `If-Modified-Since`	Revalidate by date

- OkHttp implements this for you once you give it a `Cache` — conditional requests and `304` s included
- OkHttp caches `GET` responses only

INTERVIEW TRAP

"*`no-cache` means don't cache.*" — That's `no-store`. `no-cache` stores the response but checks with the server first — cheap when the answer is a bodiless `304`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Cache in code — and its limits

```
val cacheDir = File(context.cacheDir, "http")
val client = OkHttpClient.Builder()
    .cache(Cache(cacheDir, 20L * 1024 * 1024)) // 20 MiB
    .build()

interface CatalogApi {
    @GET("catalog")
    suspend fun catalog(): CatalogDto

    // Offline fallback: cached copy or HTTP 504 — never network
    @GET("catalog")
    @Headers("Cache-Control: only-if-cached, max-stale=604800")
    suspend fun cachedCatalog(): CatalogDto
}
```

INTERVIEW TRAP

A network interceptor that rewrites every response to `max-age=3600` "to make caching work". You override the server's rules and may cache personal data. Fix the headers on the server.

FOLLOW-UP

"HTTP cache or Room?" — The HTTP cache is transparent but opaque: evictable bytes you can't query or observe. For offline-first screens the **database** is the single source of truth (Days 13–14).



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 11

Senior

Ktor & KMP · Connectivity · Cancellation & concurrency



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. The Ktor client: one networking layer for KMP

```
val client = HttpClient(OkHttp) { // Darwin engine on iOS
    expectSuccess = true
    install(ContentNegotiation) { json(json) }
    install(HttpTimeout) { requestTimeoutMillis = 15_000 }
    install(HttpRequestRetry) {
        retryOnServerErrors(maxRetries = 3)
        exponentialDelay()
    }
    defaultRequest { url("https://api.example.com/v1/") }
}

suspend fun user(id: String): UserDto =
    client.get("users/$id").body()
```

- Plugins replace interceptors: `Auth`, `HttpCache`, `Logging`, `HttpTimeout`, `HttpRequestRetry`
- Engines per platform: OkHttp or Android on Android, Darwin on iOS, CIO on the JVM



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Retrofit or Ktor?

	Retrofit + OkHttp	Ktor client
Style	Annotated interface, generated calls	Plain suspend functions with a DSL
Platforms	Android / JVM	Kotlin Multiplatform
Extensibility	Interceptors, converters, call adapters	Plugins
Best when	Android-only app, existing codebase	Shared data layer with iOS

INTERVIEW TRAP

"Ktor throws on a 404." — Only if you say so. Since Ktor 2, `expectSuccess` defaults to `false`: you get the response and must check `status`. Set `expectSuccess = true` to get `ClientRequestException` / `ServerResponseException`.

FOLLOW-UP

"How do you test either one?" — Against real HTTP responses, not mocked interfaces: `MockEngine` for Ktor, `MockWebServer` for OkHttp. Test your error mapping with a `503`, malformed JSON and a slow body.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Observing connectivity the right way

```
private fun NetworkCapabilities?.validated() =
    this?.hasCapability(NET_CAPABILITY_VALIDATED) == true

fun ConnectivityManager.onlineFlow() = callbackFlow<Boolean> {
    val callback = object : NetworkCallback() {
        override fun onCapabilitiesChanged(
            network: Network,
            caps: NetworkCapabilities,
        ) {
            trySend(caps.validated())
        }
        override fun onLost(network: Network) {
            trySend(false)
        }
    }
    trySend(getNetworkCapabilities(activeNetwork).validated())
    registerDefaultNetworkCallback(callback)
    awaitClose { unregisterNetworkCallback(callback) }
}.distinctUntilChanged()
```

- `VALIDATED` = the system actually reached the internet — not just "Wi-Fi is on" (captive portals)
- `activeNetworkInfo` / `NetworkInfo` are deprecated since API 29



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Connectivity rules

- Use the flow for **UI hints** ("You're offline") and to trigger a refresh when the network returns
- Deferrable uploads and syncs: WorkManager with a network constraint (Day 15)
- Respect metered networks: check `NET_CAPABILITY_NOT_METERED` before large downloads

INTERVIEW TRAP

"Check `isConnected` before every request." — The network can change between the check and the call, and "connected" doesn't mean "has internet". **Just make the request** and handle the failure; connectivity is a hint, not a gate.

FOLLOW-UP

"The user is offline and taps Send. What happens?" — Save it locally first, show it as pending, and let WorkManager deliver it when the network returns. Never lose user input because of the network.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Cancellation, main-safety & concurrency

```
suspend fun loadProfile(id: String): Profile = coroutineScope {
    val user = async { api.user(id) }
    val stats = async { api.stats(id) }
    Profile(user.await().toDomain(), stats.await().toDomain())
}
```

- Retrofit and Ktor `suspend` calls are **main-safe**; cancelling the coroutine **cancels the HTTP call**
- `coroutineScope`: if one request fails, the other is cancelled — no leaked work
- The user leaves the screen → `viewModelScope` is cancelled → in-flight reads stop

INTERVIEW TRAP

"`awaitAll` on 50 calls to one host runs 50 requests in parallel." — OkHttp's `Dispatcher` allows **5 concurrent async calls per host** (64 total) by default; the rest queue. Prefer a batch endpoint over tuning `maxRequestsPerHost`.

INTERVIEW TRAP

Sending a message or payment from `viewModelScope`: the user navigates away and the write is cancelled halfway. Writes that must finish belong in an application-scoped coroutine or WorkManager.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 11

→

Architect

Network module design · Resilience at scale · Evolving APIs



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Designing the network module

```
:core:network      one OkHttpClient · Json · interceptors
        ▲  newBuilder() per backend
:feature:x:data    XApi · XRemoteDataSource · DTO → model
        ▲  domain models only
:feature:x:ui      ViewModel · screens
```

```
val base = OkHttpClient.Builder()
    .cache(cache)
    .addInterceptor(userAgentInterceptor)
    .build()

val paymentsClient = base.newBuilder()
    .callTimeout(30, TimeUnit.SECONDS)
    .addInterceptor(PaymentsAuthInterceptor(tokens))
    .build()
```

- `newBuilder()` **shares** the connection pool, dispatcher and cache — different policy, same resources
- Environments come from build types or flavors, not a runtime base-URL switch in release



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Module rules

- One `Json` and one root `OkHttpClient`, provided by DI; each backend gets its own derived client and API interface
- Remote data sources return domain models or typed errors — `HttpException` never crosses the data layer
- Cross-cutting concerns live once in `:core:network`: user agent, auth, logging, metrics

INTERVIEW TRAP

Every feature builds its own `OkHttpClient` from scratch: N connection pools, N thread pools — and two `Cache` instances on the same directory corrupt each other. Derive clients, don't duplicate them.

FOLLOW-UP

"How do you see what's slow in production?" — An OkHttp `EventListener` reports DNS, connect, TLS and response timings per call; feed them into your monitoring (Day 28).



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Resilience at scale

Pattern	Protects against
Timeouts on every call	Hanging forever on a dead connection
Backoff + jitter, capped attempts	Retry storms after an outage
Honor `Retry-After` on `429` / `503`	Hammering a server that asked for time
Idempotency keys	Double charges when a retry follows a lost response
Request deduplication	Five screens fetching the same profile at once
Circuit breaker / kill switch	Calling an endpoint that is known to be down
Graceful degradation	One failed widget taking down the whole screen

INTERVIEW TRAP

Retrying at every layer: 3 attempts in the app × 3 in the gateway × 3 in the service = up to **27** requests for one tap. Decide which single layer owns retries, and keep a retry budget.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Idempotency keys in practice

```
interface PaymentsApi {
    @POST("payments")
    suspend fun pay(
        @Header("Idempotency-Key") key: String,
        @Body body: PaymentDto,
    ): PaymentResultDto
}

// One key per user intent — reused on every retry
val key = UUID.randomUUID().toString()
val result = withRetry(isTransient = ::isTransient) {
    api.pay(key, payment)
}
```

- The server stores the key with the result; a retry with the same key returns the **same** result instead of charging again
- Persist the key with the pending operation, so it survives process death and WorkManager retries

MY TAKE

Retries are a product decision, not a network detail. For each call, write down: is it idempotent, who retries it, and what the user sees meanwhile.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Evolving APIs without breaking old apps

Old app versions stay installed for years. The server must never break them, and the client must tolerate change.

```
@Serializable
enum class Status { ACTIVE, SUSPENDED, UNKNOWN }

@Serializable
data class AccountDto(
    val id: String,
    // New server values → UNKNOWN (with coerceInputValues)
    val status: Status = Status.UNKNOWN,
    // Added in a later API version: optional
    val nickname: String? = null,
)
```

INTERVIEW TRAP

The server adds a new enum value and every old app crashes on parse. Always keep a fallback: an `UNKNOWN` default for enums, a default deserializer for polymorphic types, `ignoreUnknownKeys` for objects.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. The compatibility contract

Change	Safe?
Add an optional field or a new endpoint	Yes — old clients ignore it
Add an enum value	Only if clients have a fallback
Rename or remove a field	No — add the new one, deprecate the old, remove after a sunset
Change a field's type or meaning	No — new field or new API version

- Version where it's visible: `/v2/` in the path or a version header — one strategy, applied consistently
- A **minimum supported version** endpoint plus in-app updates for the rare breaking change (Day 24)
- Contract tests or OpenAPI-generated clients catch breaks in CI, not in production

SENIOR ANSWER IN ONE LINE

"Assume every call can be slow, fail or run twice: bounded timeouts, retries only where safe, typed errors at the boundary, and an API contract that tolerates change."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 11 · ARCHITECT

Day 11 — cheat sheet

- `DELETE` and `PUT` are idempotent, `POST` is not; idempotent ≠ safe.
- `401` → refresh the token; `403` → don't. `204` has no body.
- Retrofit describes the API, OkHttp executes it. `suspend` is native since 2.6 — no call adapter needed.
- kotlinx.serialization with `ignoreUnknownKeys`; Gson ignores Kotlin defaults and nullability.
- Application interceptors run once per call; network interceptors once per network hop.
- `readTimeout` is a gap, not a total — bound the whole call with `callTimeout` or `withTimeout`.
- Retry only transient failures on idempotent calls: backoff + jitter, honor `Retry-After`, idempotency keys for writes.
- Map exceptions to a sealed `ApiError` at the data layer; always rethrow `CancellationException`.
- `no-cache` ≠ `no-store`. HTTP cache for resources; Room as the source of truth for offline-first.
- Connectivity is a hint, not a gate. One root `OkHttpClient`, derived with `newBuilder()`.

Day 12 is next → Network Layer, Part 2: security & scale. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700